

AF_PACKET Mode Architecture Guide

Purpose: Complete technical guide to understanding how the AF_PACKET mode IDS pipeline works, from packet capture to ML predictions.

Table of Contents

- 1. [High-Level Overview](#)
- 2. [Architecture Components](#)
- 3. [Key Functions Breakdown](#)
- 4. [Command-Line Usage](#)

High-Level Overview

What is AF_PACKET Mode?

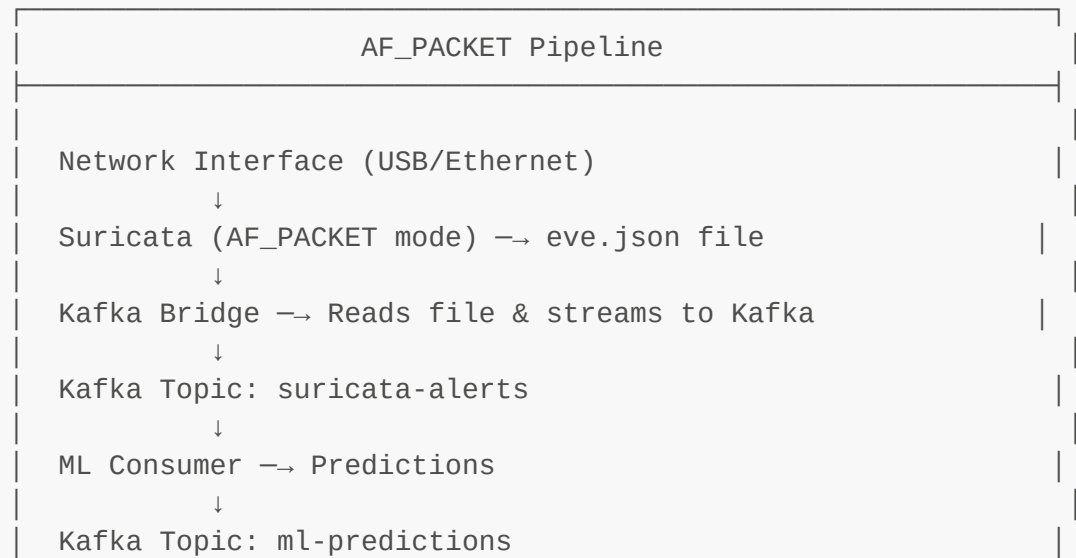
AF_PACKET is a Linux kernel-level packet capture mechanism that allows applications to receive raw network packets from network interfaces. It's the standard way to capture network traffic in Linux, used by tools like **tcpdump** and **Wireshark**.

Why We Use It

- ✔ **Universal Compatibility:** Works with ANY network interface (USB, Ethernet, WiFi, virtual)
- ✔ **No Special Hardware:** No need for expensive Intel/Mellanox NICs
- ✔ **Simpler Setup:** Doesn't require kernel module compilation or interface binding
- ✔ **Great for Testing:** Perfect for development and testing environments
- ✔ **USB Adapter Support:** Works perfectly with USB Ethernet adapters

Architecture at a Glance

PROF



Key Characteristic: Uses a **file-based intermediary** (eve.json) between Suricata and Kafka.

Architecture Components

1. Network Interface Layer

Component: USB Ethernet Adapter (or any network interface)

Example: `enx00e04c36074c` (USB), `eth0` (Ethernet), `wlan0` (WiFi)

Configuration Requirements:

- **IP Address:** Static IP in capture subnet (e.g., 192.168.100.1/24)
 - **Promiscuous Mode:** ENABLED - captures ALL packets, not just those destined for this interface
 - **State:** UP - interface must be active
 - **Firewall:** Configured to allow traffic but not route it
-

2. AF_PACKET Capture Layer

Component: Linux Kernel AF_PACKET Socket

Managed By: Suricata IDS

How AF_PACKET Works:

```
// Conceptual C code showing AF_PACKET socket creation

// 1. Create raw socket
int sock = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));

// 2. Bind to specific interface
struct sockaddr_ll addr;
addr.sll_family = AF_PACKET;
addr.sll_ifindex = if_nametoindex("enx00e04c36074c");
bind(sock, (struct sockaddr *)&addr, sizeof(addr));

// 3. Receive packets
while (1) {
    recvfrom(sock, buffer, BUFFER_SIZE, 0, NULL, NULL);
    // Process packet...
}
```

AF_PACKET Features Used:

- **PACKET_FANOUT:** Distributes packets across multiple threads for parallel processing

- **PACKET_RX_RING**: Memory-mapped ring buffer for efficient packet reception (reduces system calls)
- **Zero-copy**: Packets read directly from kernel memory without copying

Configuration in Suricata:

```
# /etc/suricata/suricata.yaml

af-packet:
  - interface: enx00e04c36074c
    threads: 4                # Parallel capture threads
    cluster-id: 99            # PACKET_FANOUT cluster ID
    cluster-type: cluster_flow # Load balance by flow
    defrag: yes               # Reassemble fragmented packets
    use-mmap: yes             # Use memory-mapped ring buffer
    ring-size: 2048           # Ring buffer size (packets)
    block-size: 32768         # Memory block size (bytes)
```

Performance Tuning:

- **Threads**: Set to number of CPU cores for parallel processing
- **Ring Size**: Larger = can handle bursts better, but uses more memory
- **Cluster Type**:
 - **cluster_flow** - Same flow goes to same thread (maintains order)
 - **cluster_cpu** - Round-robin distribution
 - **cluster_qm** - Queue mapping

3. Suricata IDS Engine

Component: Suricata IDS (Open Source IDS/IPS)

Version: 7.0.x

Role: Deep packet inspection, threat detection, event logging

What Suricata Does:

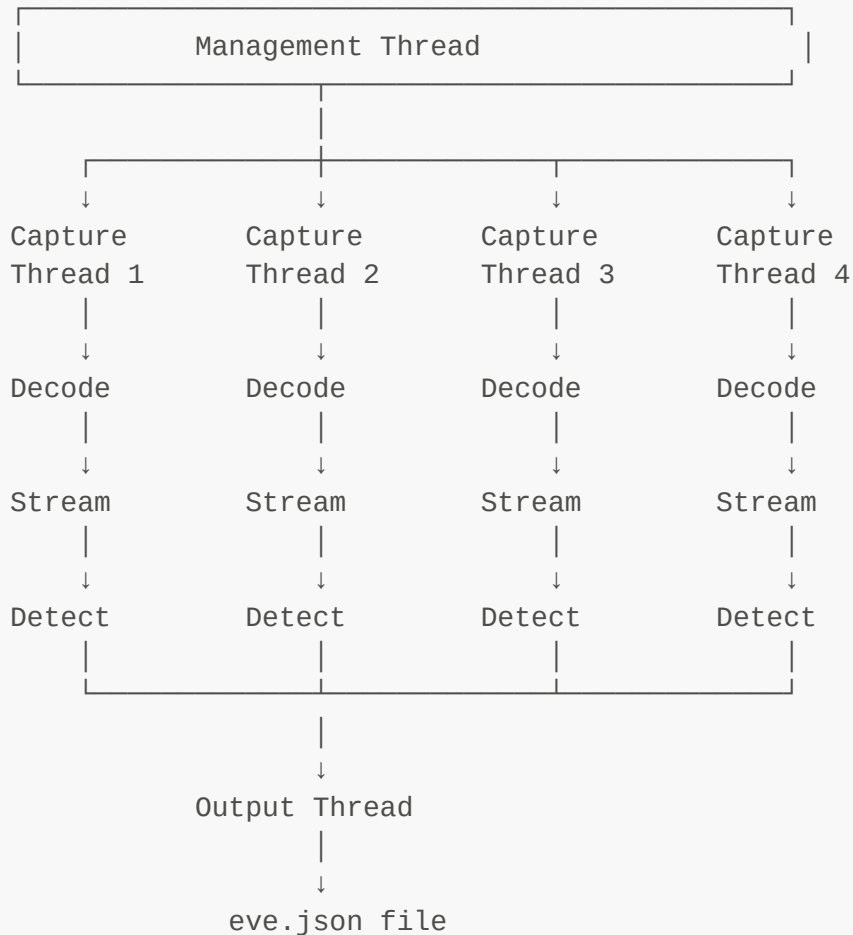
1. **Packet Capture**: Receives raw packets from AF_PACKET socket
2. **Protocol Parsing**: Decodes packets (Ethernet → IP → TCP/UDP → Application)
3. **Flow Tracking**: Maintains state for TCP connections and UDP sessions
4. **Signature Matching**: Compares packets against threat signatures (Emerging Threats rules)
5. **Anomaly Detection**: Detects protocol violations and suspicious patterns
6. **Event Logging**: Writes detection events to eve.json file

Multi-Stage Processing Pipeline:

```
Packet → Decode → Stream → Detection → Logging
      ↓         ↓         ↓         ↓
```

Key Suricata Features:

a) Multi-Threading Architecture:



PROF

b) Flow Management:

```
# Conceptual flow tracking

flows = {} # Track all active connections

def process_packet(packet):
    flow_key = (src_ip, dst_ip, src_port, dst_port, protocol)

    if flow_key not in flows:
        # New flow
        flows[flow_key] = {
            'start_time': packet.timestamp,
            'packets': 0,
            'bytes': 0,
```

```

        'state': 'NEW'
    }

    # Update flow
    flows[flow_key]['packets'] += 1
    flows[flow_key]['bytes'] += packet.length

    # Track TCP state
    if packet.tcp_flags & SYN:
        flows[flow_key]['state'] = 'SYN'
    elif packet.tcp_flags & ACK:
        flows[flow_key]['state'] = 'ESTABLISHED'

```

c) Rule Matching Engine:

Example Suricata Rule:

```

alert tcp any any -> any any (msg:"Possible SQL Injection";
    content:"UNION SELECT"; nocase; sid:1000001;)

```

Matching Process:

1. Check if packet is TCP ✓
2. Check source/dest match (any) ✓
3. Look for content "UNION SELECT" in payload
4. Match is case-insensitive
5. If found → Generate alert with sid:1000001

Suricata Configuration File (/etc/suricata/suricata.yaml):

```

# Key sections

# Capture interface
af-packet:
  - interface: enx00e04c36074c
    threads: 4

# Threading
threading:
  set-cpu-affinity: yes
  cpu-affinity:
    - management-cpu-set:
        cpu: [ 0 ]
    - receive-cpu-set:
        cpu: [ 1 ]
    - worker-cpu-set:
        cpu: [ 2, 3, 4, 5 ]

# Output
outputs:

```

```

- eve-log:
  enabled: yes
  filetype: regular
  filename: /var/log/suricata/eve.json
  types:
    - alert:
      payload: yes
      payload-buffer-size: 4kb
      payload-printable: yes
    - flow:
      enabled: yes
    - http:
      enabled: yes
    - dns:
      enabled: yes
    - tls:
      enabled: yes

```

Eve.json Event Format:

```

{
  "timestamp": "2025-10-25T14:23:45.123456+0000",
  "flow_id": 1234567890,
  "event_type": "alert",
  "src_ip": "192.168.100.50",
  "src_port": 54321,
  "dest_ip": "93.184.216.34",
  "dest_port": 80,
  "proto": "TCP",
  "alert": {
    "action": "allowed",
    "gid": 1,
    "signature_id": 2100498,
    "rev": 7,
    "signature": "GPL ATTACK_RESPONSE id check returned root",
    "category": "Potentially Bad Traffic",
    "severity": 2
  },
  "http": {
    "hostname": "example.com",
    "url": "/admin/login.php",
    "http_method": "POST",
    "protocol": "HTTP/1.1",
    "status": 200,
    "length": 1234
  },
  "payload":
    "504f5354202f61646d696e2f6c6f67696e2e70687020485454502f312e310d0a...",
  "payload_printable": "POST /admin/login.php HTTP/1.1\r\n..."
}

```

4. Eve.json File (Intermediate Storage)

Component: JSON log file

Location: `/var/log/suricata/eve.json`

Purpose: Temporary storage of Suricata events before Kafka ingestion

Why File-Based in AF_PACKET Mode?:

- Suricata in AF_PACKET mode doesn't have built-in Kafka output
- File provides decoupling: Suricata and Kafka can run independently
- If Kafka goes down, events are buffered in file (won't be lost)
- Easier debugging: Can inspect raw events in file

File Characteristics:

- **Format:** Newline-delimited JSON (one event per line)
- **Rotation:** Logrotate handles file rotation (prevents unbounded growth)
- **Permissions:** Root-only access (contains sensitive network data)
- **Growth Rate:** Depends on traffic volume (typically 100-1000 MB/day)

Logrotate Configuration (`/etc/logrotate.d/suricata`):

```
/var/log/suricata/eve.json {
    daily                # Rotate daily
    rotate 7             # Keep 7 days of logs
    compress             # Gzip old logs
    delaycompress        # Don't compress most recent
    missingok            # Don't error if file missing
    notifempty           # Don't rotate if empty
    create 0640 root root # New file permissions
    postrotate
        /bin/kill -HUP $(cat /var/run/suricata.pid) # Signal Suricata
    to reopen file
    endscript
}
```

PROF

5. Kafka Bridge (File-to-Stream Connector)

Component: Python script `suricata_kafka_bridge.py`

Role: Streams eve.json events to Kafka in real-time

Why This Component Exists:

- Bridges file-based output (Suricata) to streaming platform (Kafka)
- Provides real-time streaming without modifying Suricata
- Handles Kafka connection failures gracefully

- Adds monitoring and statistics

Key Features:

a) File Tailing:

- Opens file and seeks to end (`seek(0, 2)`)
- Reads new lines as they appear
- Polls every 100ms for new data
- Detects log rotation by monitoring inode

b) Log Rotation Handling:

```
# When logrotate runs:
# 1. eve.json renamed to eve.json.1
# 2. New eve.json created
# 3. Suricata receives HUP signal, reopens file

# Bridge detects this:
old_inode = os.stat('eve.json').st_ino
# ... time passes, rotation occurs ...
new_inode = os.stat('eve.json').st_ino

if old_inode != new_inode:
    # File changed, reopen
    file.close()
    file = open('eve.json', 'r')
```

c) Kafka Producer Configuration:

```
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',

    # Serialization
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    key_serializer=lambda k: k.encode('utf-8'),

    # Reliability
    acks='all',           # Wait for all replicas to acknowledge
    retries=3,            # Retry failed sends

    # Performance
    batch_size=16384,     # Batch messages for efficiency
    linger_ms=10,         # Wait 10ms for more messages to batch
    compression_type='gzip', # Compress messages

    # Buffering
    buffer_memory=33554432, # 32MB buffer
)
```

d) Graceful Shutdown:

```
import signal

def signal_handler(signum, frame):
    print("Received shutdown signal...")
    bridge.shutdown()
    sys.exit(0)

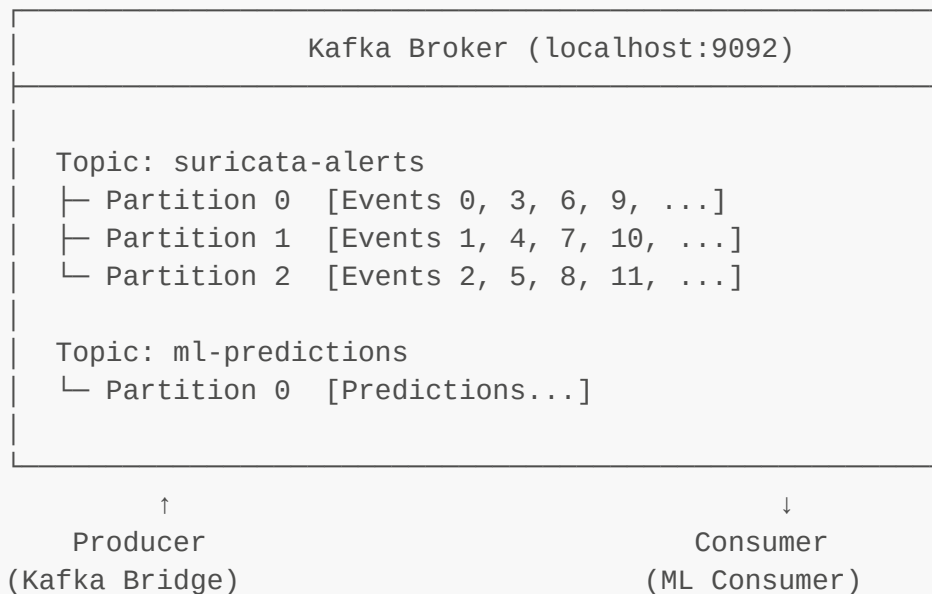
# Register signal handlers
signal.signal(signal.SIGINT, signal_handler)  # Ctrl+C
signal.signal(signal.SIGTERM, signal_handler) # kill command
```

6. Apache Kafka (Message Broker)

Component: Apache Kafka 3.6.0

Role: Distributed streaming platform for event transport

Architecture:



Why Kafka?:

- **✓ Decoupling:** Producers and consumers are independent
- **✓ Scalability:** Partitions enable parallel processing
- **✓ Durability:** Messages persisted to disk
- **✓ Replay:** Can re-process historical events
- **✓ Multiple Consumers:** Multiple systems can read same events

Topic Configuration:

```
# Topic: suricata-alerts
kafka-topics.sh --create \
  --topic suricata-alerts \
  --partitions 3 \
  --replication-factor 1 \
  --config retention.ms=86400000      # 24 hour retention
  --config segment.bytes=104857600   # 100MB segments
  --config compression.type=gzip      # Compress on disk

# Topic: ml-predictions
kafka-topics.sh --create \
  --topic ml-predictions \
  --partitions 1 \
  --replication-factor 1
```

Message Format:

```
Key: flow_id (string) - Used for partitioning
Value: Full JSON event (gzip compressed)
Timestamp: Event timestamp
Headers: [event_type, severity, etc.]
```

Kafka Operations:

a) Message Production (Bridge):

```
# Kafka bridge sends message
producer.send(
    topic='suricata-alerts',
    key=str(event['flow_id']), # Same flow always goes to same
    partition
    value=event,               # Full JSON event
    timestamp_ms=event_timestamp
)
```

b) Message Consumption (ML Consumer):

```
# ML consumer reads messages
consumer = KafkaConsumer(
    'suricata-alerts',
    bootstrap_servers='localhost:9092',
    group_id='ml-consumer-group',
    auto_offset_reset='latest', # Start from most recent
```

```

enable_auto_commit=True,      # Auto-commit offsets
value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

for message in consumer:
    event = message.value
    process_event(event)

```

c) Offset Management:

Topic: suricata-alerts, Partition 0

0	1	2	3	4	5	6	... Offsets
---	---	---	---	---	---	---	-------------

↑
Consumer Position (offset 3)

Consumer commits: "I've processed up to offset 3"

If consumer crashes and restarts: Resume from offset 4

7. ML Consumer (Machine Learning Inference)

Component: Python script `ml_kafka_consumer.py`

Role: Real-time threat classification using machine learning

What It Does:

1. **Consumes Events:** Reads from `suricata-alerts` Kafka topic
2. **Feature Extraction:** Extracts 65 CICIDS2017 features from events
3. **ML Inference:** Runs pre-trained models (Random Forest, LightGBM, etc.)
4. **Adaptive Ensemble:** Combines predictions from multiple models
5. **Threat Scoring:** Calculates confidence scores and risk levels
6. **Output:** Publishes enriched events to `ml-predictions` topic

PROF

Key Functions Breakdown

1. Initialization & Checks

```
print_header()
```

- Displays colorful banner with mode information
- Shows that script is running in AF_PACKET mode
- Makes it clear this is USB-compatible

```
check_root()
```

- Verifies script is run with **sudo**
- **Why needed:** Interface configuration and Suricata require root

```
load_config()
```

- Loads settings from **dppk_suricata_ml_pipeline/config/pipeline.conf**
- Reads variables like:
 - **NETWORK_INTERFACE** - Which interface to capture on
 - **KAFKA_BOOTSTRAP_SERVERS** - Kafka connection details
 - **SURICATA_LOG_DIR** - Where logs are stored

```
check_dependencies()
```

- Validates required software is installed:
 - **Suricata** - IDS engine
 - **Kafka** - Message broker
 - **tcpreplay** - For PCAP replay
 - **Python3** - For ML consumer
- Exits with error if any dependency missing

```
check_interface()
```

- Verifies network interface exists
- Checks if **NETWORK_INTERFACE** is configured
- Lists available interfaces if specified one not found
- Example check:

```
if ! ip link show "$NETWORK_INTERFACE" > /dev/null 2>&1; then
    echo "Interface not found!"
    ip link show # Show what's available
fi
```

—
PROF

2. Component Startup Functions

Each function starts one pipeline component and verifies it's running:

`start_kafka()`

What it does:

1. Checks if Kafka already running (`pgrep -f "kafka.Kafka"`)
2. If not running, calls `02_setup_kafka.sh`
3. Waits 3 seconds for startup
4. Verifies Kafka process exists
5. Exits with error if startup failed

Why 3 second wait: Kafka needs time to initialize Zookeeper connection

`start_suricata()`

What it does:

1. Checks if Suricata already running
2. Calls `03_start_suricata_afpacket.sh`
3. Starts Suricata with AF_PACKET capture on specified interface
4. Waits 3 seconds
5. Verifies Suricata process running with `--af-packet` flag

Process check:

```
if pgrep -f "suricata.*--af-packet" > /dev/null; then
    echo "✓ Suricata running"
fi
```

`start_kafka_bridge()`

What it does:

1. Checks if bridge script already running
2. Calls `04_start_kafka_bridge.sh`
3. Starts `suricata_kafka_bridge.py` in background
4. Bridge reads `eve.json` and streams to Kafka
5. Waits 2 seconds
6. Verifies bridge process exists

Why needed in AF_PACKET mode: Suricata writes to file, not directly to Kafka

`start_ml_consumer()`

What it does:

1. Checks if ML consumer already running
2. Calls `05_start_ml_consumer.sh` in background (&)

3. Starts `ml_kafka_consumer.py` daemon
4. Waits 3 seconds for initialization
5. Verifies ML consumer process running

Background execution (&): Allows script to continue while consumer runs

3. Status & Monitoring Functions

`show_status()`

What it does:

Displays comprehensive system status with color-coded indicators:

1. Kafka Status

```
if pgrep -f "kafka.Kafka" > /dev/null; then
    echo "✓ Kafka: Running"
else
    echo "✗ Kafka: Not running"
fi
```

2. Suricata Status

- Shows if running
- Displays PID
- Shows which interface it's capturing on

```
SURICATA_PID=$(pgrep -f "suricata.*-af-packet")
echo "PID: $SURICATA_PID"
echo "Interface: $NETWORK_INTERFACE"
```

3. Kafka Bridge Status

- Shows if bridge is streaming events

4. ML Consumer Status

- Shows if ML predictions are running

5. Network Interface Status

- Shows if interface is UP
- Checks if promiscuous mode enabled (required for packet capture)

```
if ip link show "$NETWORK_INTERFACE" | grep -q "PROMISC"; then
    echo "✓ Promiscuous mode enabled"
```

fi

Color codes:

-  Green ✓ = Running/OK
-  Red ✗ = Not running
-  Yellow ⚠ = Warning

view_logs()

What it does:

Interactive log viewer with 4 options:

1. Suricata logs - IDS engine logs

```
tail -f /var/log/suricata/suricata.log
```

2. ML consumer logs - ML prediction logs

```
tail -f dpdk_suricata_ml_pipeline/logs/ml/ml_consumer.log
```

3. Kafka bridge logs - File-to-Kafka streaming logs

```
tail -f dpdk_suricata_ml_pipeline/logs/kafka_bridge.log
```

4. All logs - Monitors all simultaneously

```
tail -f suricata.log ml_consumer.log kafka_bridge.log
```

Uses **tail -f**: Follows log files in real-time (like watching live feed)

4. Shutdown Functions

stop_all()

What it does:

Gracefully stops all components **in correct order** (reverse of startup):

Shutdown sequence:

- | | |
|-----------------|------------------|
| 1. ML Consumer | ← Top of chain |
| 2. Kafka Bridge | ← Middle |
| 3. Suricata | ← Source |
| 4. Kafka | ← Infrastructure |

Why this order?

- ML Consumer depends on Kafka events → stop first
- Bridge depends on Suricata logs → stop second
- Suricata generates events → stop third
- Kafka is infrastructure → stop last

Implementation:

```
# Stop ML consumer
if pgrep -f "ml_kafka_consumer.py" > /dev/null; then
    pkill -f "ml_kafka_consumer.py"
    echo "✓ ML consumer stopped"
fi

# Stop Kafka bridge
if pgrep -f "suricata_kafka_bridge.py" > /dev/null; then
    pkill -f "suricata_kafka_bridge.py"
    echo "✓ Kafka bridge stopped"
fi

# Stop Suricata
if pgrep -f "suricata" > /dev/null; then
    pkill -f "suricata"
    sleep 2 # Wait for clean shutdown
    echo "✓ Suricata stopped"
fi

# Stop Kafka (calls stop_all.sh script)
if pgrep -f "kafka.Kafka" > /dev/null; then
    bash "${PIPELINE_SCRIPTS}/stop_all.sh"
    echo "✓ Kafka stopped"
fi
```

sleep 2 after Suricata: Gives time for Suricata to flush buffers and close files properly

5. Additional Functions

`setup_external_capture()`

What it does:

1. Calls `00_setup_external_capture.sh`
2. Configures network interface for capture
3. Sets up isolated network (192.168.100.0/24)
4. Enables promiscuous mode
5. Configures firewall rules

When to use: First-time setup or after reboot

`replay_traffic()`

What it does:

1. Calls `05_replay_traffic.sh`
2. Replays PCAP files to test IDS
3. Useful for testing without real attacks

6. Interactive Menu System

`show_menu()`

Displays options:

```
===== MENU =====
1) Start Complete Pipeline (Kafka + Suricata + ML)
2) Start Kafka Only
3) Start Suricata Only (AF_PACKET)
4) Start ML Consumer Only
5) Start Kafka Bridge Only
6) Replay Traffic (PCAP)
7) Check Status
8) View Logs
9) Setup External Capture 🌐
10) Stop All Services
0) Exit
=====
```

`main()` - Menu Loop

How it works:

```
while true; do
    show_menu
    read -p "Enter choice [0-10]: " choice

    case $choice in
        1) # Start complete pipeline
```

```

        start_kafka
        start_suricata
        start_kafka_bridge
        start_ml_consumer
        show_status
        ;;
    2) start_kafka ;;
    3) start_suricata ;;
    # ... etc
    0) exit 0 ;;
esac

read -p "Press Enter to continue..."
done

```

Menu loop benefits:

- User doesn't need to remember commands
- Can perform multiple operations without restarting
- Visual feedback after each action

Command-Line Usage (Non-Interactive)

Script supports direct execution for automation:

```

# Start complete pipeline
sudo ./run_afpacket_mode.sh start

# Individual components
sudo ./run_afpacket_mode.sh kafka      # Start Kafka only
sudo ./run_afpacket_mode.sh suricata   # Start Suricata only
sudo ./run_afpacket_mode.sh ml         # Start ML consumer only
sudo ./run_afpacket_mode.sh bridge     # Start Kafka bridge only

# Utilities
sudo ./run_afpacket_mode.sh status     # Check status
sudo ./run_afpacket_mode.sh logs       # View logs
sudo ./run_afpacket_mode.sh setup      # Setup external capture
sudo ./run_afpacket_mode.sh stop       # Stop everything

```

Why useful:

- Automation scripts can call it
 - Can integrate with systemd or cron
 - Scripting workflows
-